

MATH 8090: Stationary Processes: Properties, Mean, and Covariance Functions

Whitney Huang, Clemson University

8/31-9/4/2026

Contents

Examples of i.i.d. Noise	1
Examples realizations of white noise processes	3
MA(1) processes	9
AR(1) processes	13
Random walk	16
Gaussian process	19
Different covaraince functions (kernels)	19
Generate one sample from each Gaussian Process with different kernels	20

Examples of i.i.d. Noise

```
# Set the length of the time series
T = 200
t <- 1:T

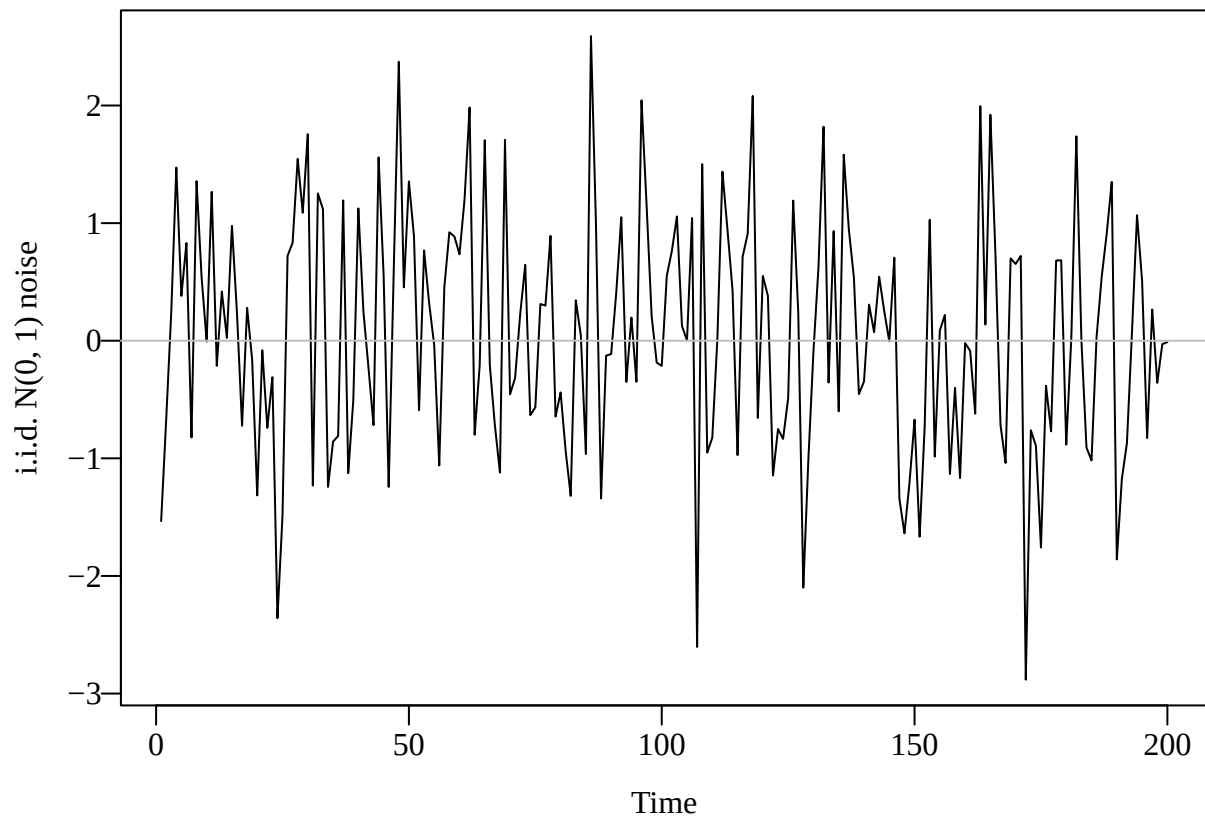
## i.i.d. normal

# Generate 100 independent N(0, 1) random variables
normal_iid <- rnorm(T)

# Set graphical parameters
par(las = 1, mgp = c(2, 0.5, 0), mar = c(3.6, 3.6, 0.8, 0.6),
    family = "serif")

# Plot the i.i.d. normal noise over time
plot(t, normal_iid, type = "l", las = 1,
     xlab = "Time", ylab = "i.i.d. N(0, 1) noise")

# Add a horizontal reference line at zero
abline(h = 0, col = "gray")
```



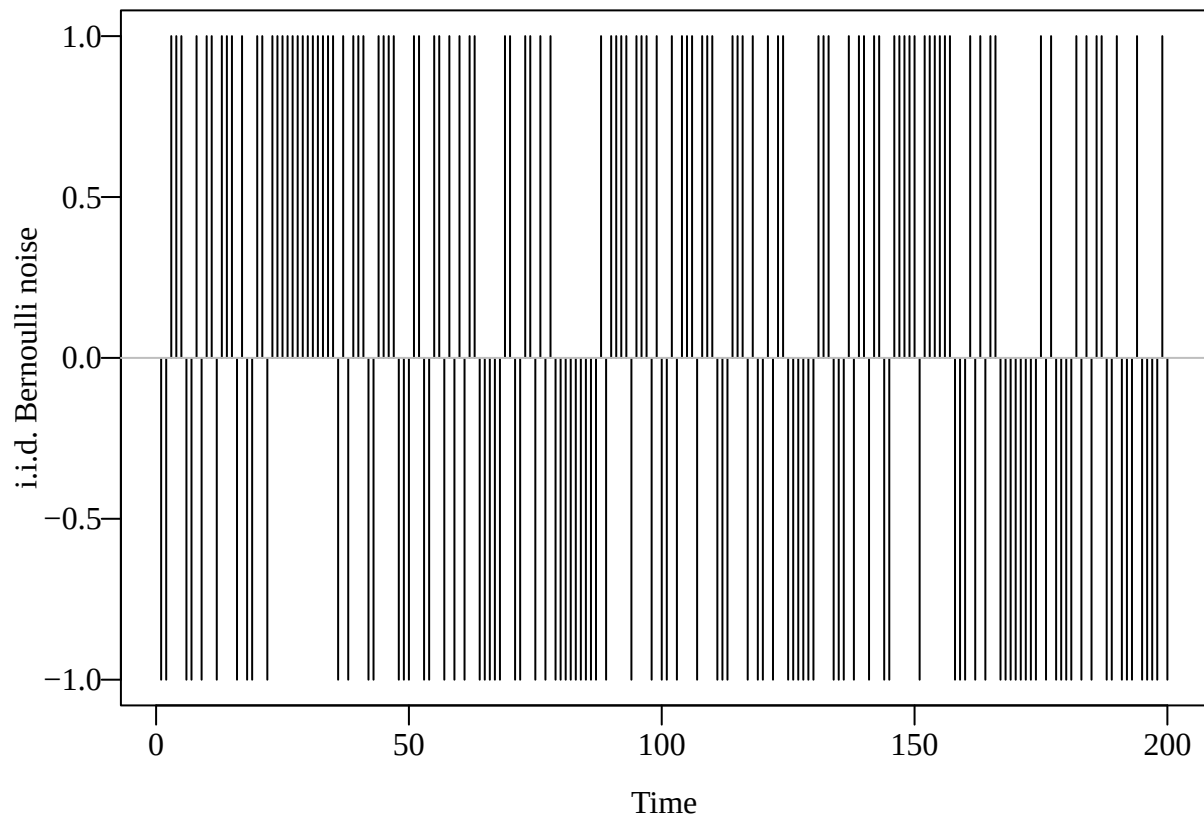
```
## i.i.d. Binary

# Generate 100 independent Bernoulli(0.5) random variables
ber_iid <- replicate(T, rbinom(1, 1, 0.5))

# Transform Bernoulli outcomes from {0, 1} to {-1, 1}
ber_iid <- ifelse(ber_iid == 0, -1, 1)

# Plot the i.i.d. binary noise over time
plot(t, ber_iid, type = "h", las = 1,
     xlab = "Time", ylab = "i.i.d. Bernoulli noise")

# Add a horizontal reference line at zero
abline(h = 0, col = "gray")
```



Examples realizations of white noise processes

If Z_t is a *white noise* process, then its mean and variance are constants and uncorrelated in time

Note: here we do not require the sequence follow the same distribution.

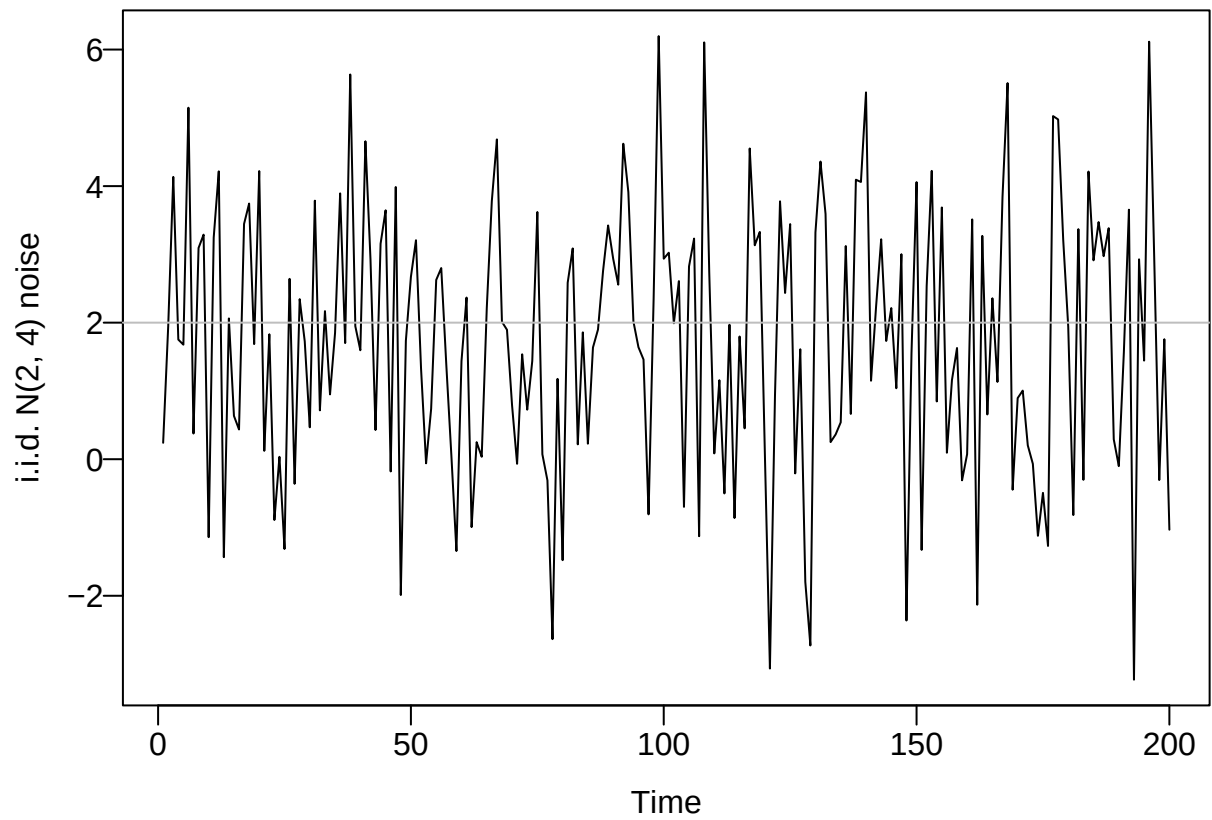
```
# Set the length of the time series
T = 200
t <- 1:T

# Generate Gaussian white noise with mean 2 and variance 4
WN1 <- rnorm(n = T, mean = 2, sd = 2)

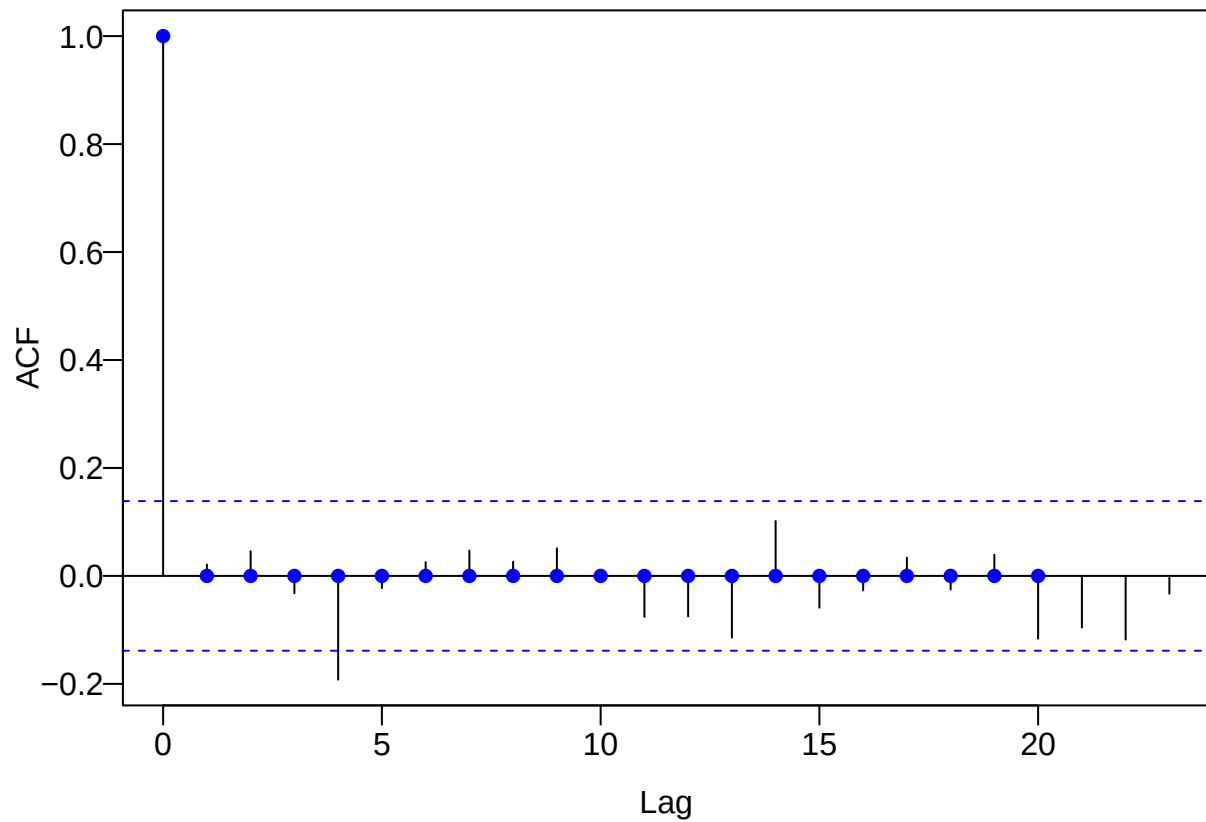
# Set graphical parameters
par(las = 1, mgp = c(2, 0.5, 0), mar = c(3.6, 3.6, 0.8, 0.6))

# Plot the Gaussian white noise
plot(t, WN1, type = "l", xlab = "Time", ylab = "i.i.d. N(2, 4) noise")

# Add a horizontal reference line at the mean
abline(h = 2, col = "gray")
```



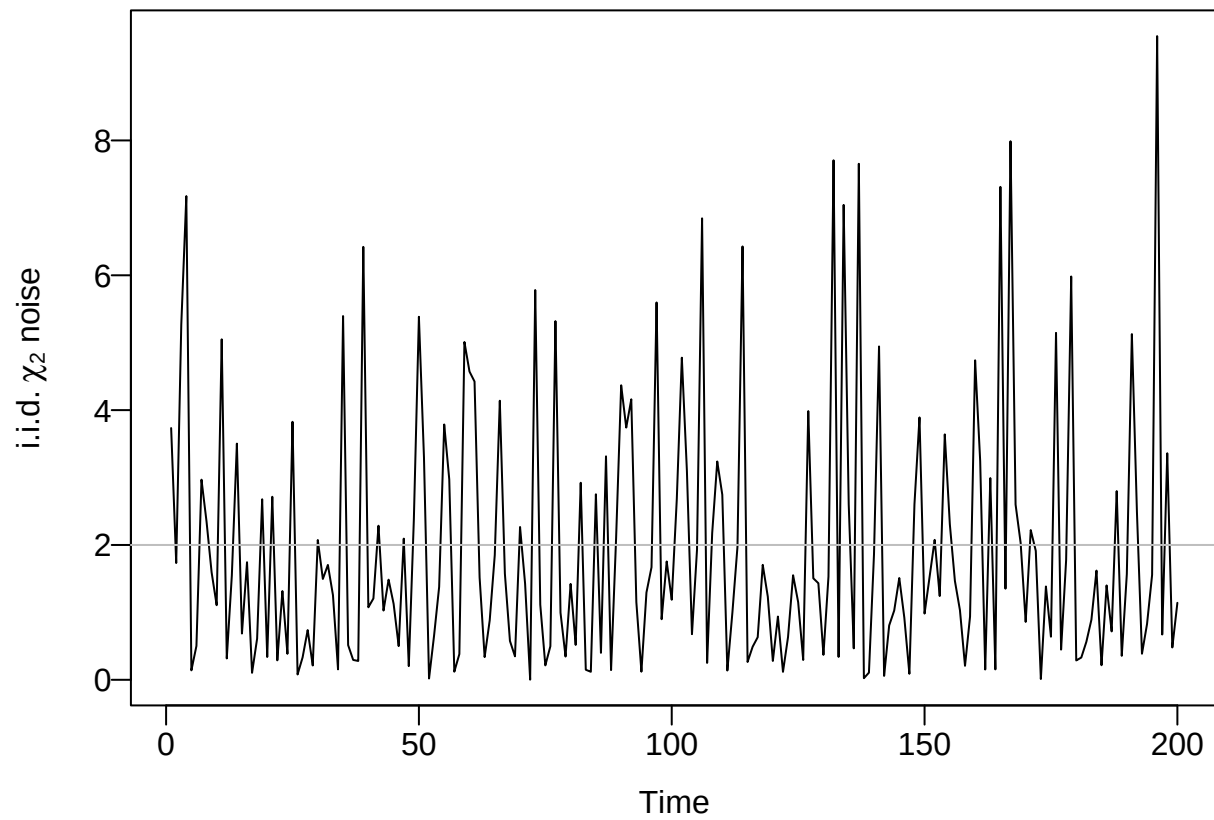
```
# Plot the sample autocorrelation function (ACF)  
acf(WN1)  
  
# Add the theoretical ACF of white noise: 1 at lag 0 and 0 otherwise  
points(0:20, c(1, rep(0, 20)), pch = 16, col = "blue")
```



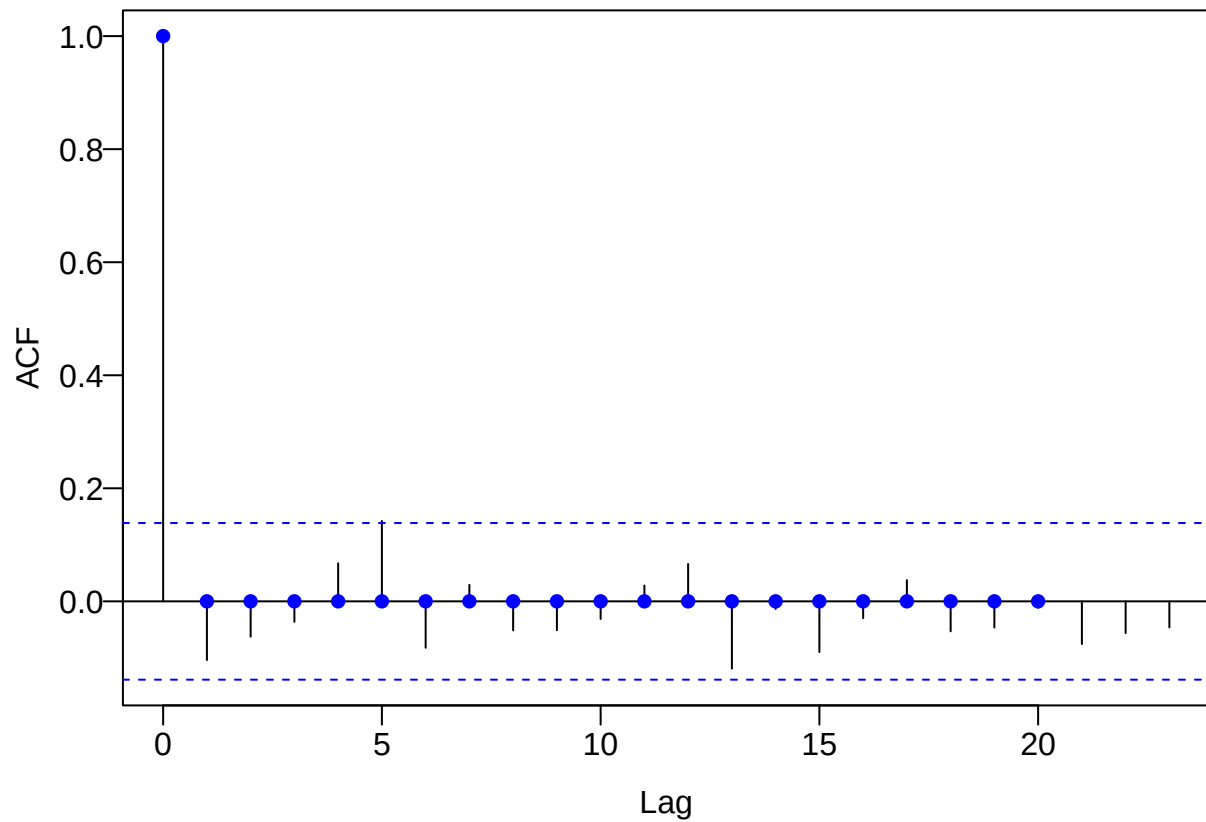
```
# Generate chi-squared white noise with 2 degrees of freedom
WN2 <- rchisq(n = T, df = 2)

# Plot the chi-squared white noise
plot(t, WN2, type = "l", xlab = "Time", ylab = expression(paste("i.i.d. ", chi[2], " noise")))

# Add a horizontal reference line at the mean
abline(h = 2, col = "gray")
```



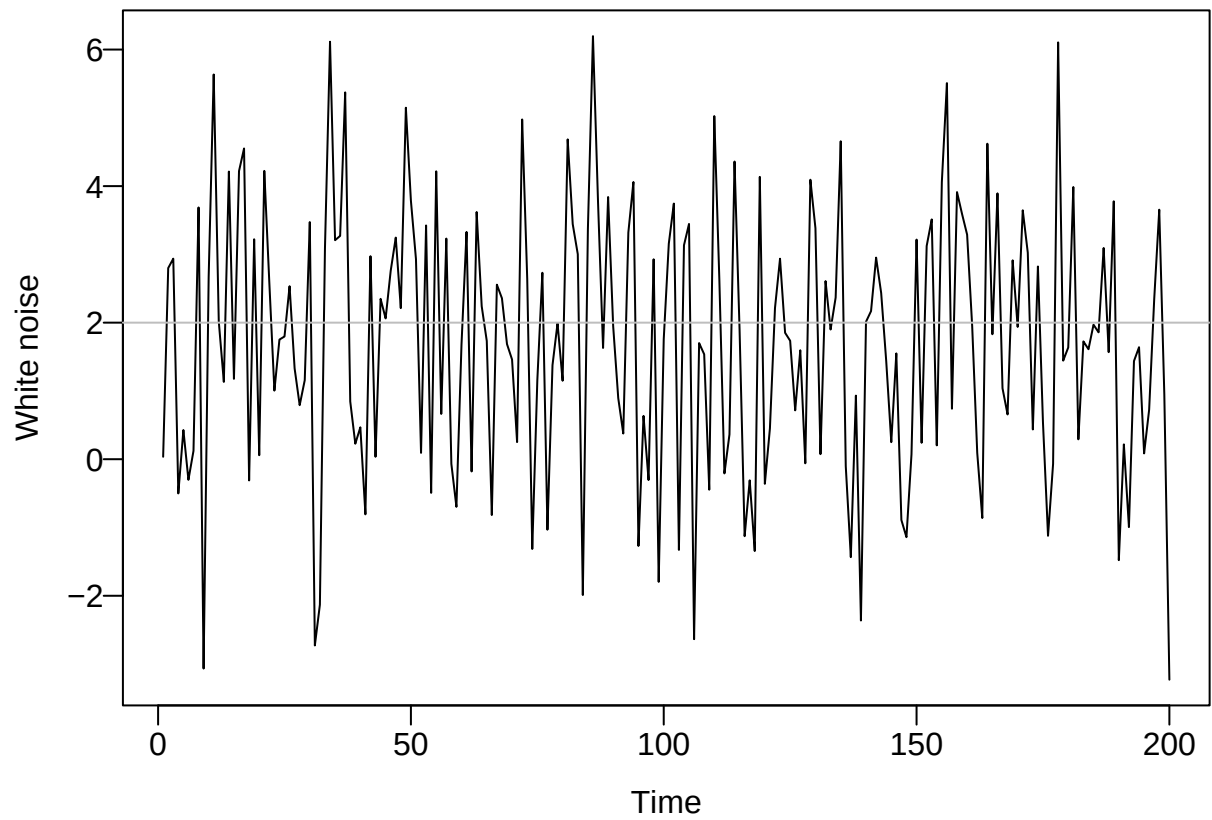
```
# Plot the sample autocorrelation function (ACF)  
acf(WN2)  
  
# Add the theoretical ACF of white noise  
points(0:20, c(1, rep(0, 20)), pch = 16, col = "blue")
```



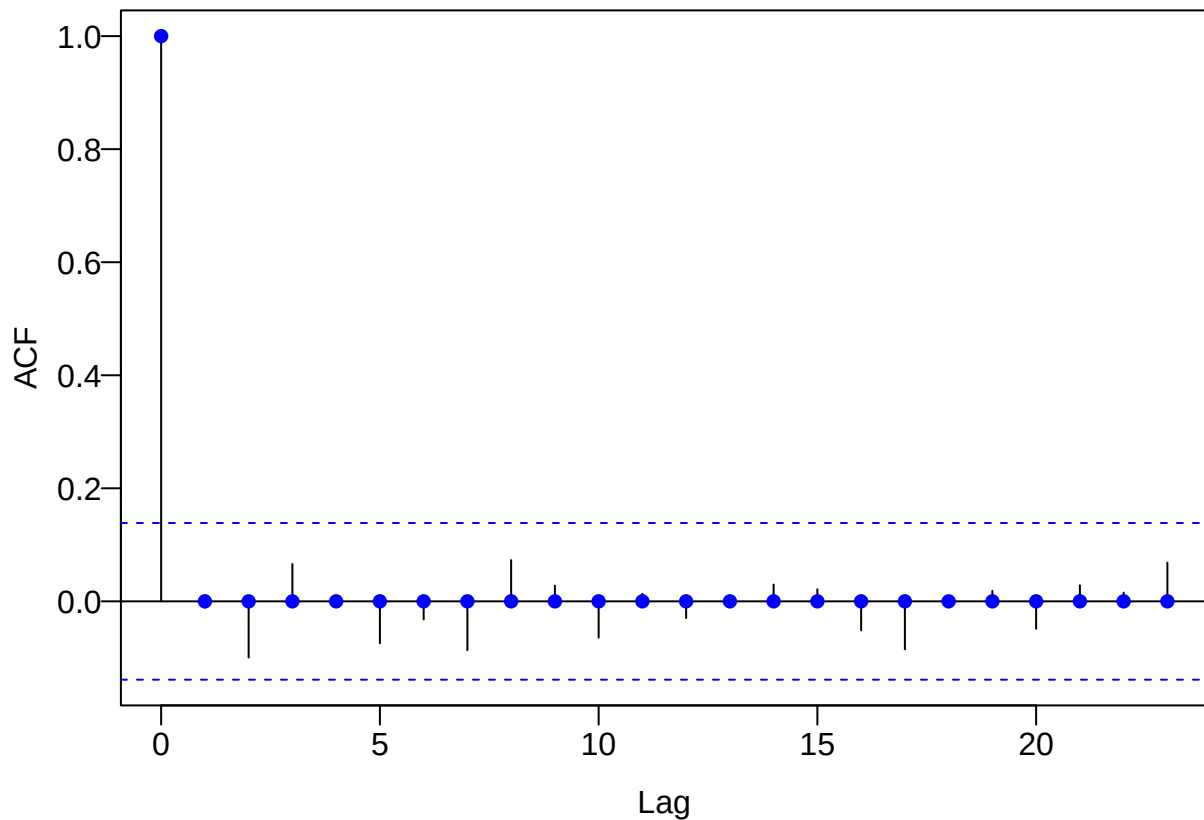
```
# Combine the two white noise series and randomly permute the observations
WN3 <- c(WN1, WN2)[sample(1:200)]

# Plot the resulting white noise series
plot(1:200, WN3, type = "l", xlab = "Time", ylab = expression(paste("White noise")))

# Add a horizontal reference line at the common mean
abline(h = 2, col = "gray")
```



```
# Plot the sample autocorrelation function (ACF)  
acf(WN3)  
  
# Add the theoretical ACF of white noise  
points(0:23, c(1, rep(0, 23)), pch = 16, col = "blue")
```



MA(1) processes

$$\eta_t = Z_t + \theta Z_{t-1},$$

where $Z \sim \text{WN}(0, \sigma^2)$.

```
# Set the length of the time series
T = 100
t <- 1:T

# Generate Gaussian white noise with extra observations for burn-in
z <- rnorm(110)

# Specify four values of the MA(1) parameter theta
theta <- c(0.25, 1, -1, -0.25)

# Set graphical parameters and arrange plots in a 2 x 1 layout
par(las = 1, mgp = c(2, 0.5, 0), mar = c(3.6, 3.6, 0.8, 0.6), mfrow = c(2, 1))

# Simulate and examine an MA(1) process for each value of theta
for (i in 1:4){

  # Generate MA(1):  $X_t = Z_t + \theta Z_{t-1}$ , and remove burn-in observations
  MA1 <- filter(z, sides = 1, c(1, theta[i]))[-(1:10)]

  # Plot the simulated MA(1) time series
  plot(t, MA1, type = "l", xlab = "Time", ylab = "MA(1)")
}
```

```

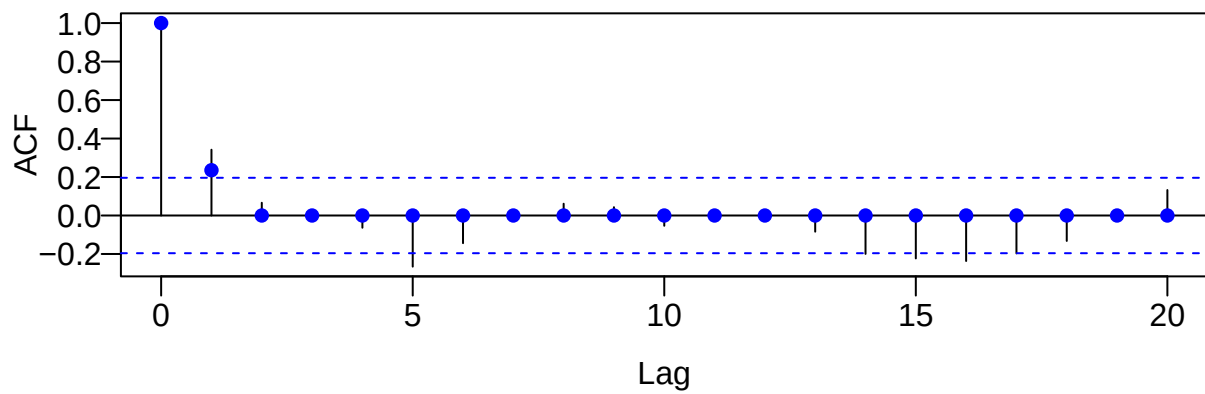
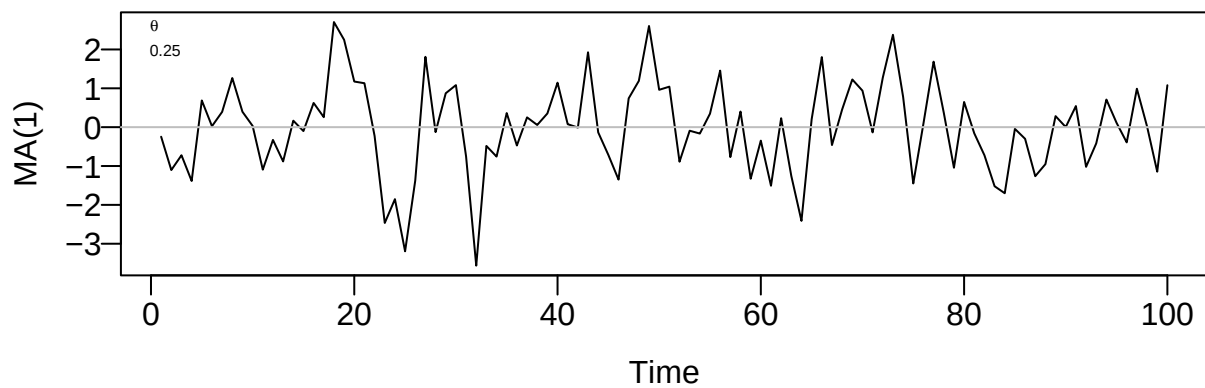
# Add a horizontal reference line at the mean
abline(h = 0, col = "gray")

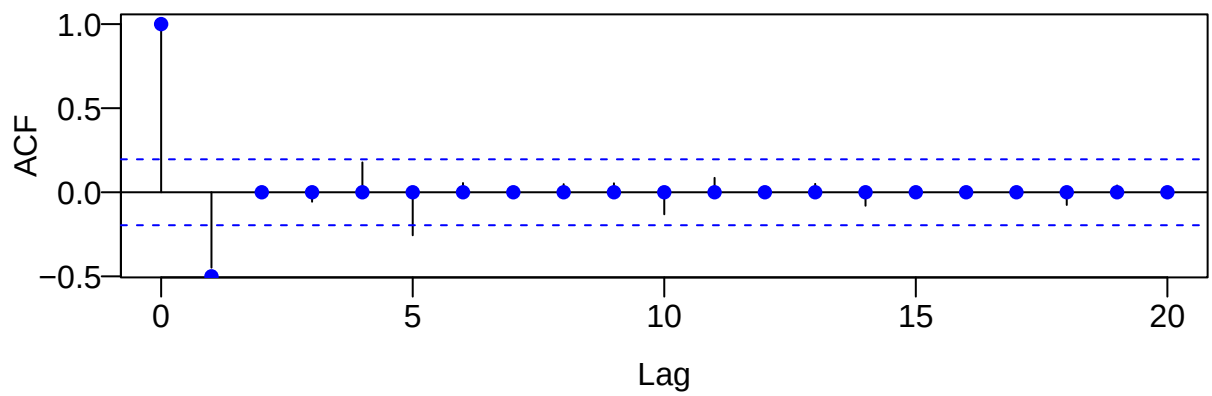
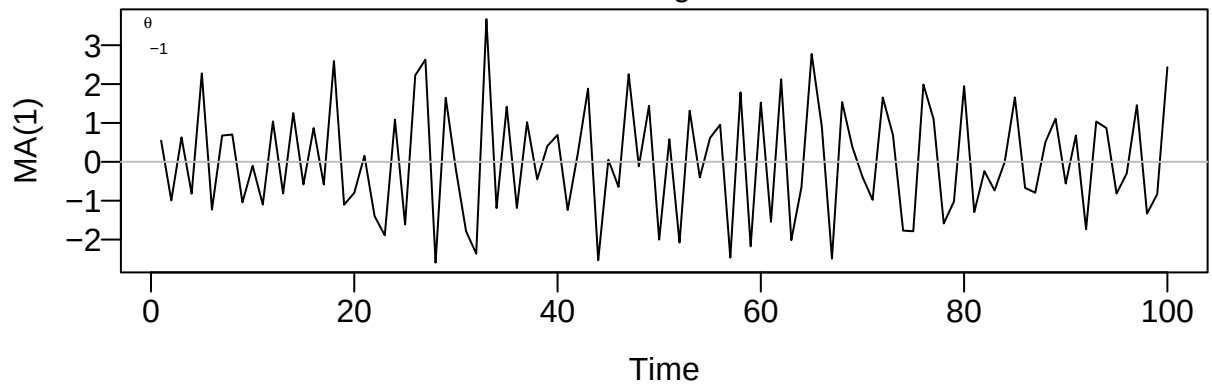
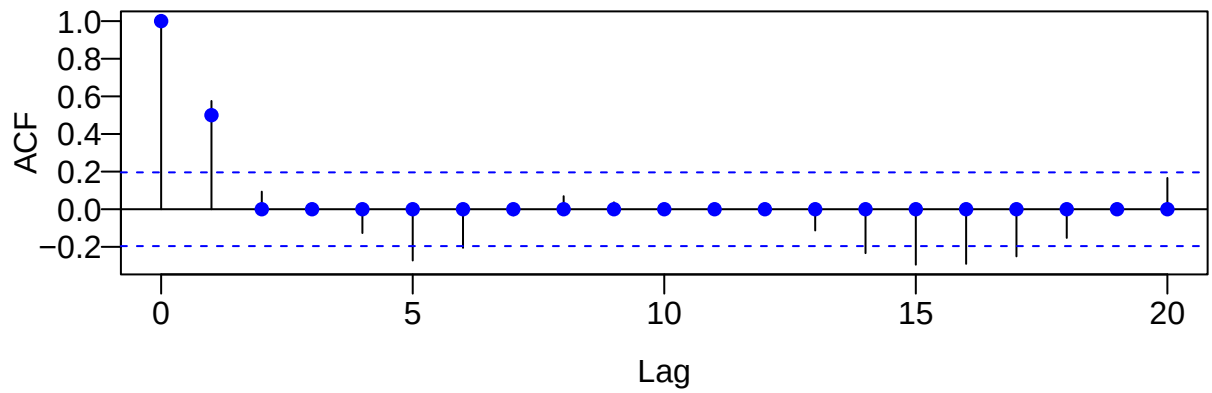
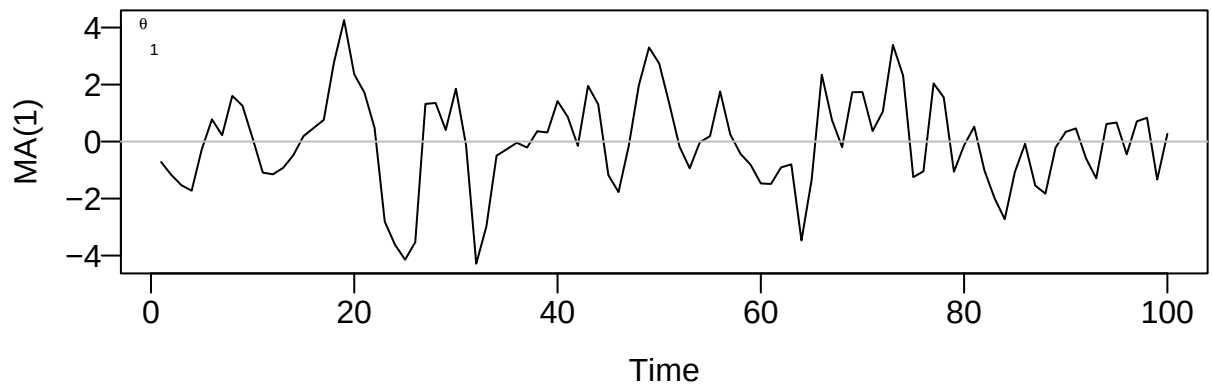
# Display the value of theta used in the simulation
legend("topleft", legend = theta[i], title = expression(theta),
      bty = "n", cex = 0.5)

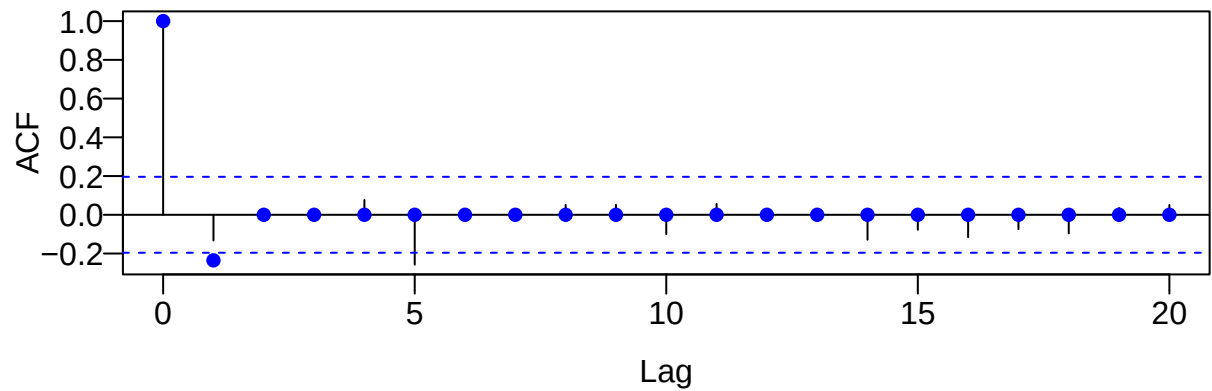
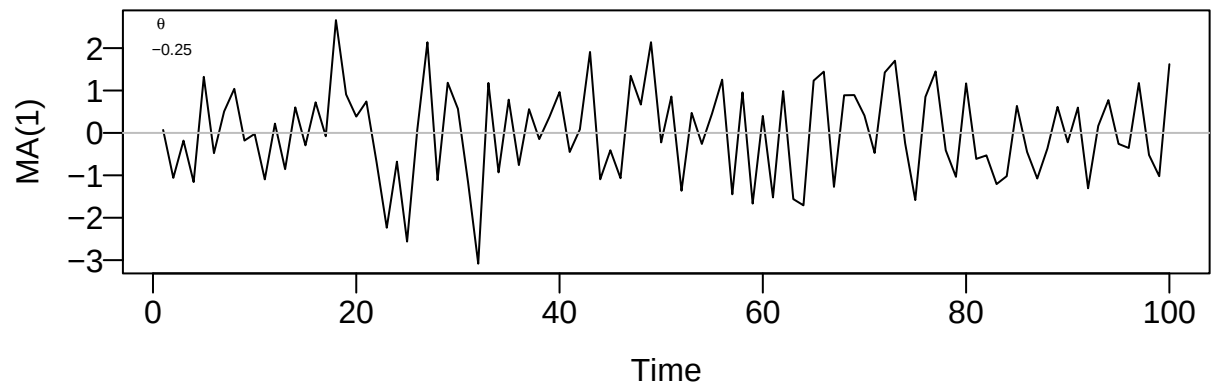
# Plot the sample autocorrelation function (ACF)
acf(MA1)

# Add the theoretical ACF: rho(1) = theta/(1 + theta^2) and rho(h) = 0 for h > 1
points(0:20, c(1, theta[i] / (1 + theta[i]^2), rep(0, 19)),
      pch = 16, col = "blue")
}

```





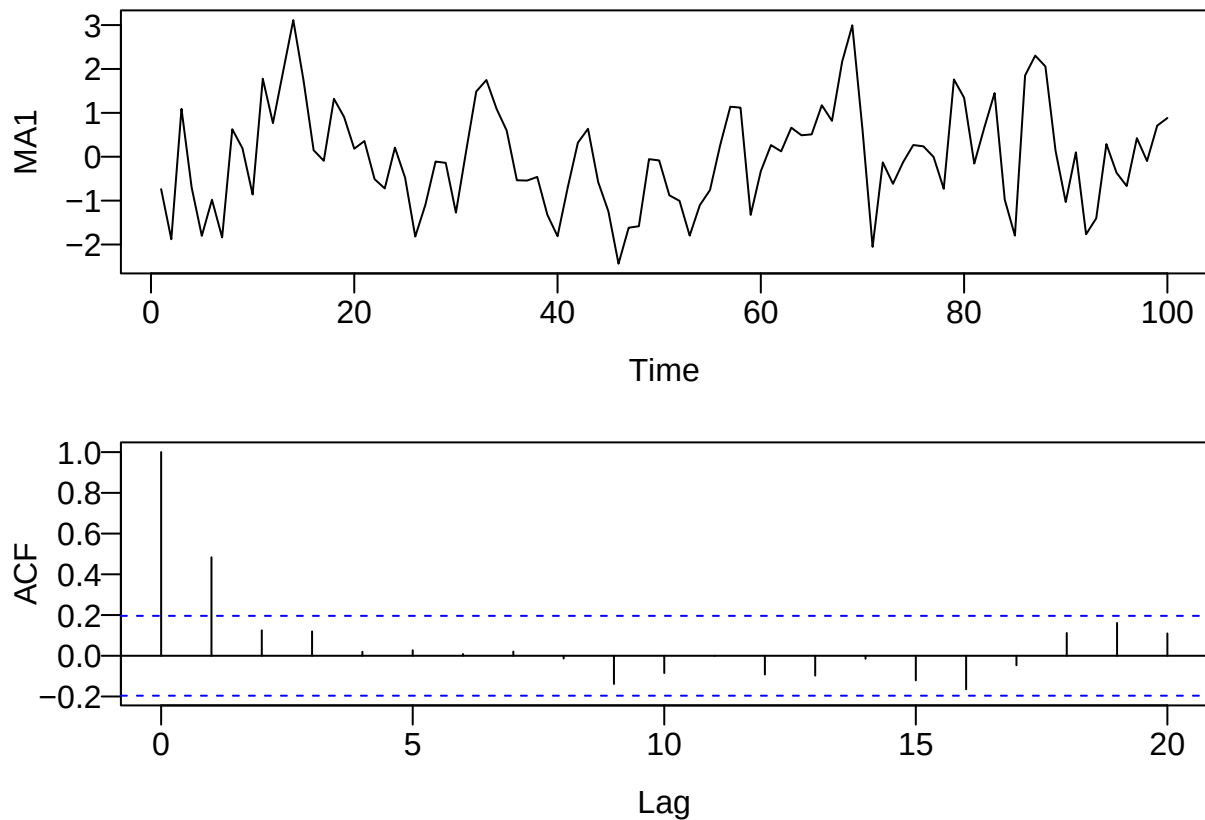


```
## Another way to simulate an MA(1) process

# Simulate an MA(1) process with theta = 0.5 using arima.sim()
MA1 <- arima.sim(n = 100, list(ma = c(0.5)))

# Plot the simulated MA(1) time series
plot(MA1)

# Plot the sample ACF
acf(MA1)
```



AR(1) processes

$$\eta_t = \phi\eta_{t-1} + Z_t,$$

where $|\rho| < 1$ is a constant and η_s and Z_t are uncorrelated for all $s < t \Rightarrow$ future noise is uncorrelated with the current value.

```
# Specify three values of the AR(1) parameter phi
phi <- c(0.25, 0.9, -0.5)

# Set graphical parameters and arrange plots in a 2 x 1 layout
par(las = 1, mgp = c(2, 0.5, 0), mar = c(3.6, 3.6, 0.8, 0.6),
    mfrow = c(2, 1), family = "serif")

# Simulate and examine an AR(1) process for each value of phi
for (i in 1:3){

  # Simulate AR(1):  $X_t = \phi X_{t-1} + Z_t$ 
  AR1 <- arima.sim(n = 100, list(ar = c(phi[i])))

  # Plot the simulated AR(1) time series
  plot(t, AR1, type = "l", xlab = "Time",
       ylab = paste("AR(1), rho = ", phi[i]))

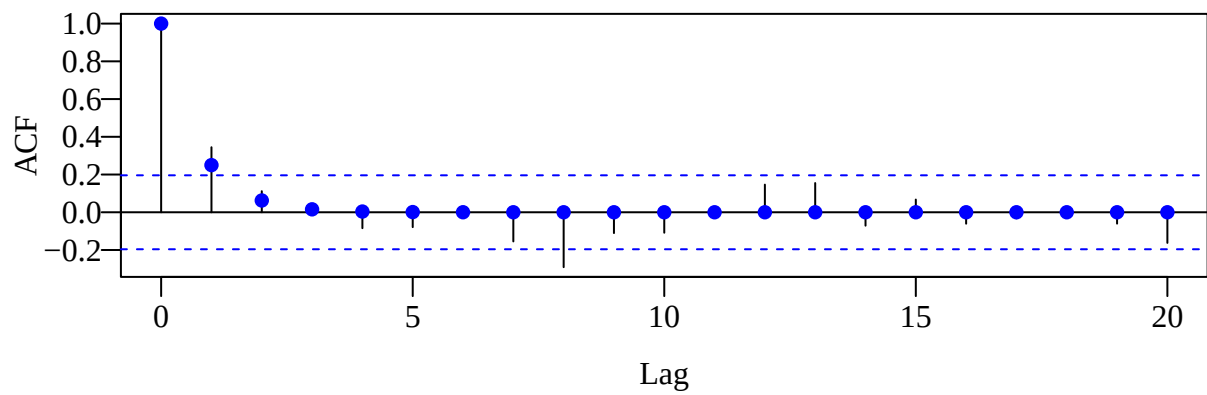
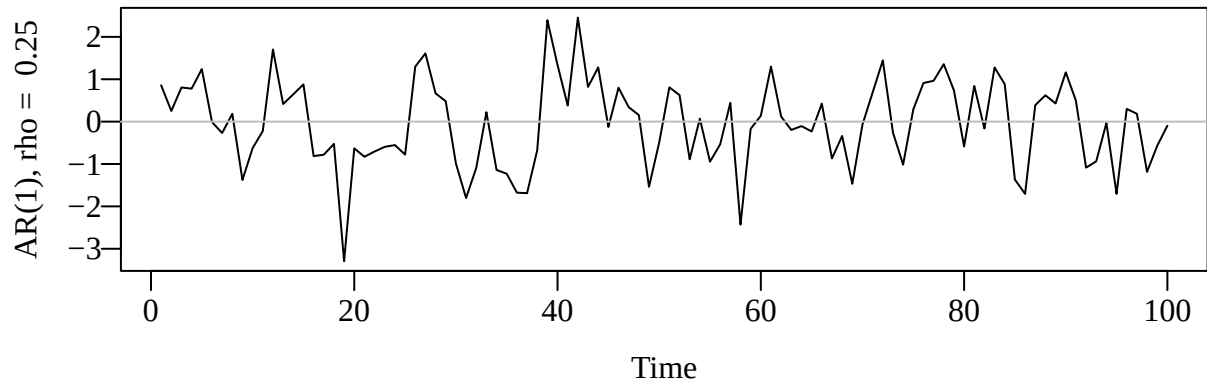
  # Add a horizontal reference line at the mean
  abline(h = 0, col = "gray")
}
```

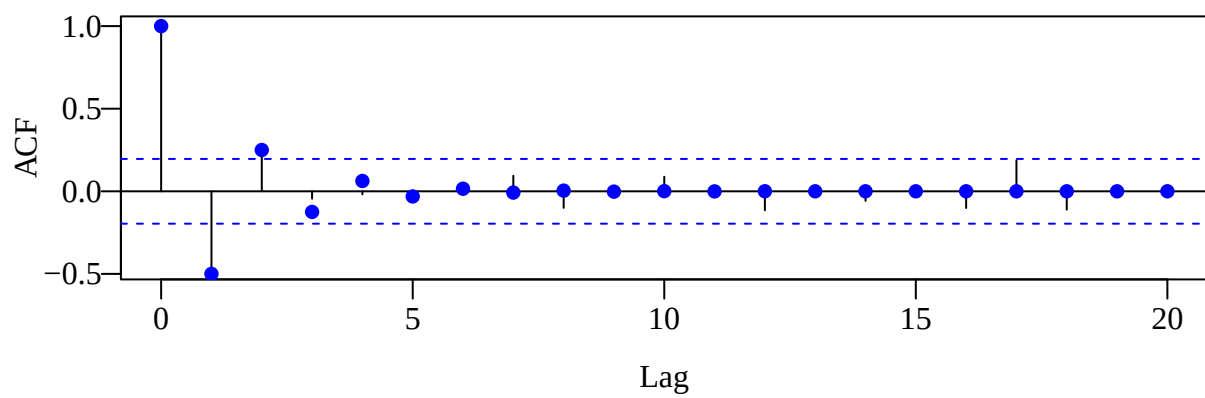
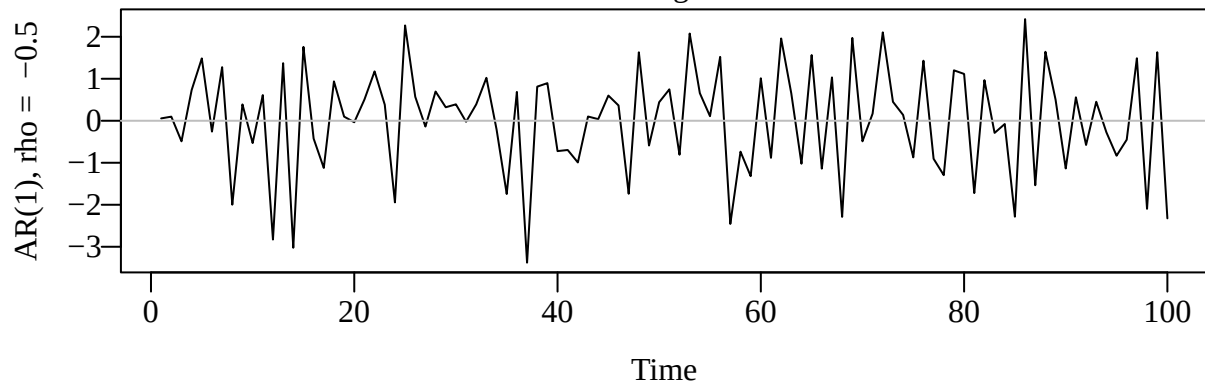
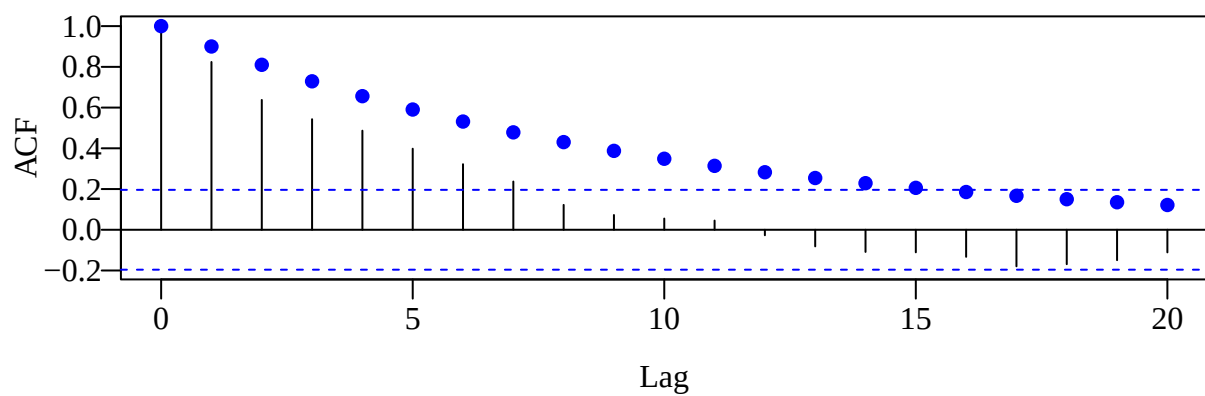
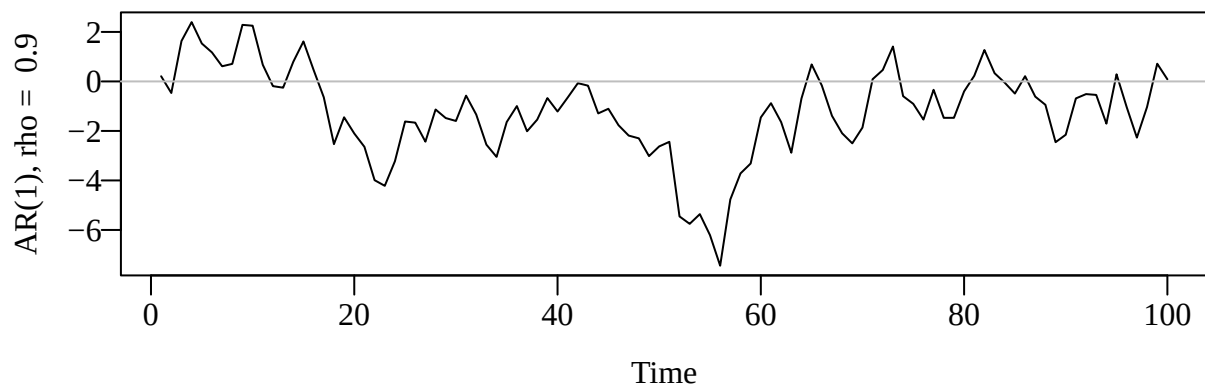
```

# Plot the sample autocorrelation function (ACF)
acf(AR1)

# Add the theoretical ACF:  $\rho(h) = \phi^h$ 
points(0:20, phi[i]^(0:20), pch = 16, col = "blue")
}

```





Random walk

$$\eta_t = \sum_{s=1}^t Z_s.$$

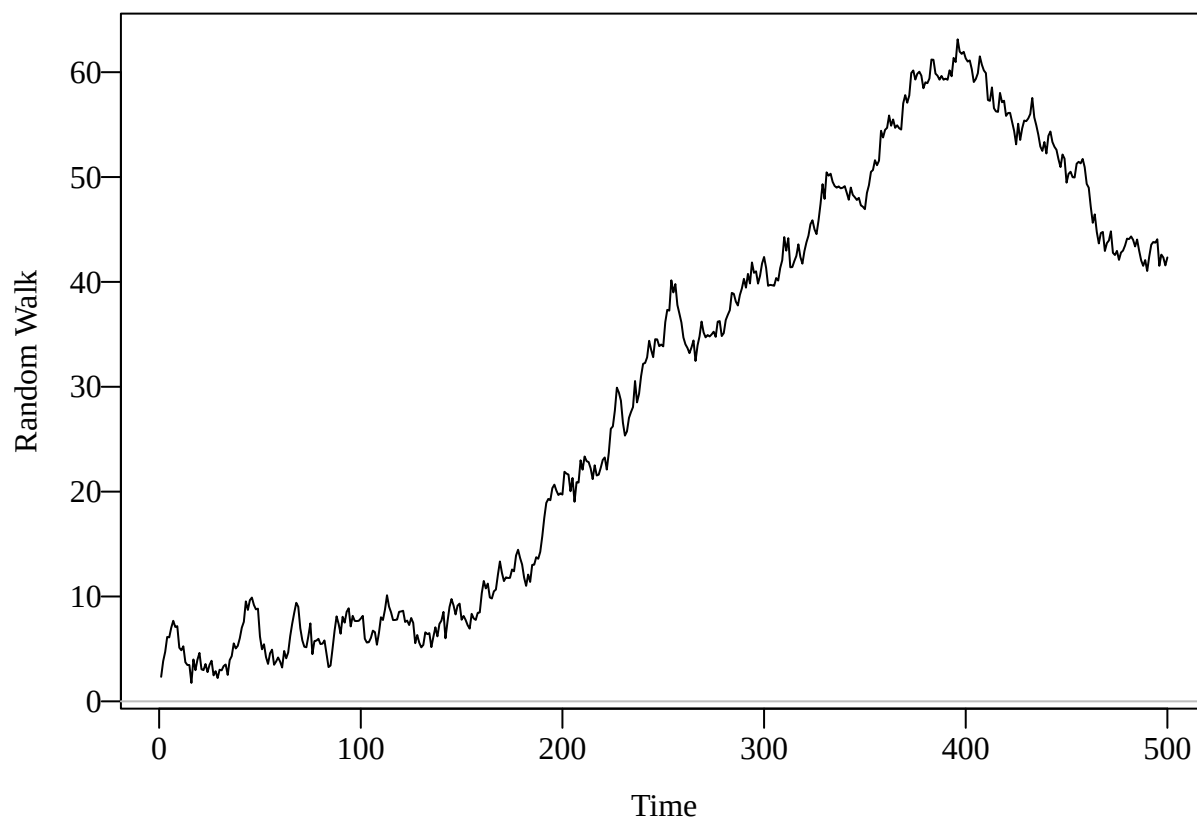
```
# Set graphical parameters
par(las = 1, mgp = c(2, 0.5, 0), mar = c(3.6, 3.6, 0.8, 0.6),
    family = "serif")

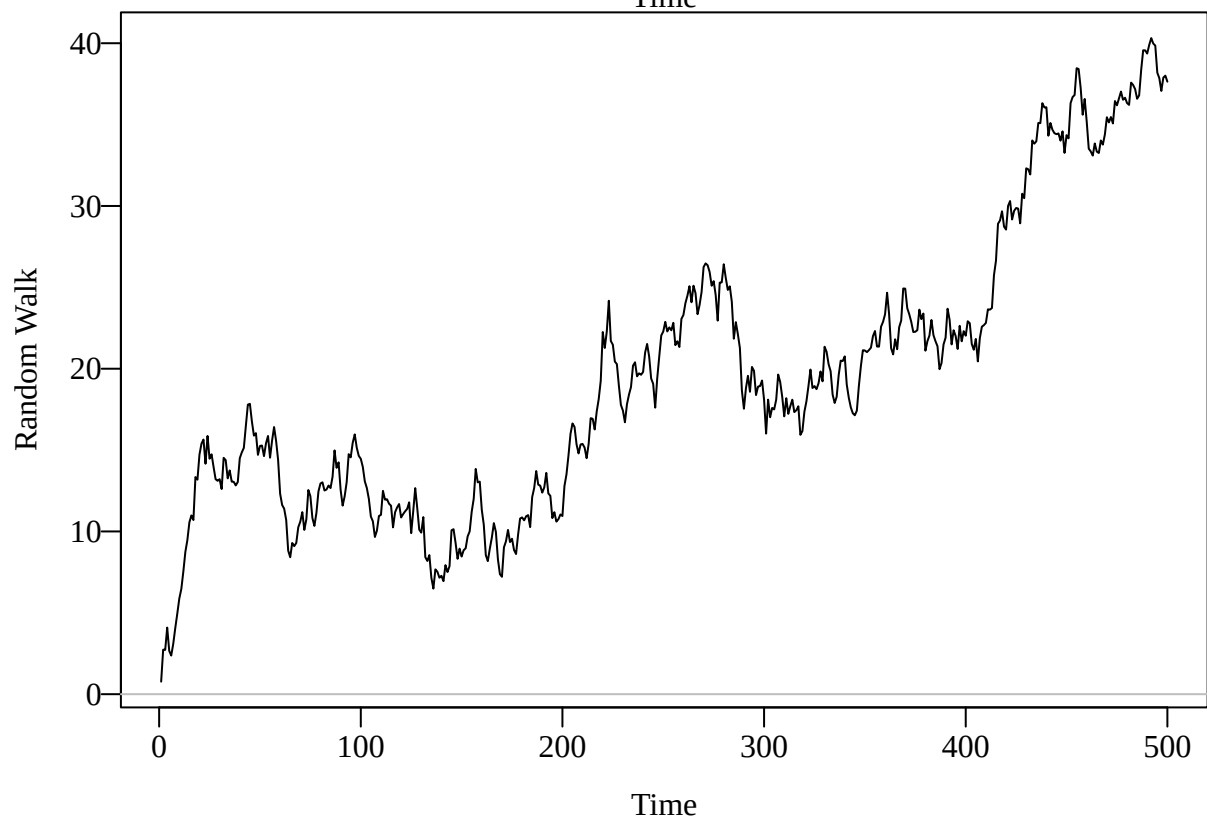
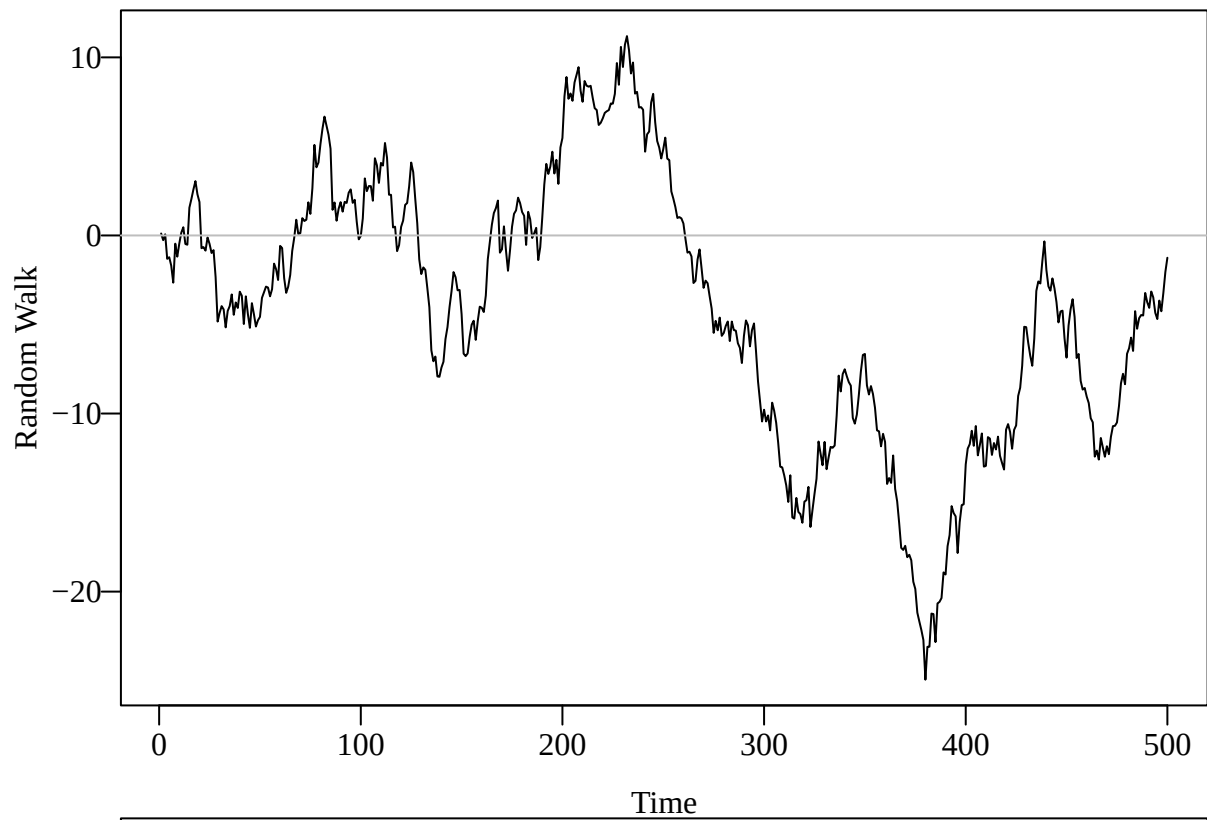
# Generate and plot five realizations of a random walk
for (i in 1:5){

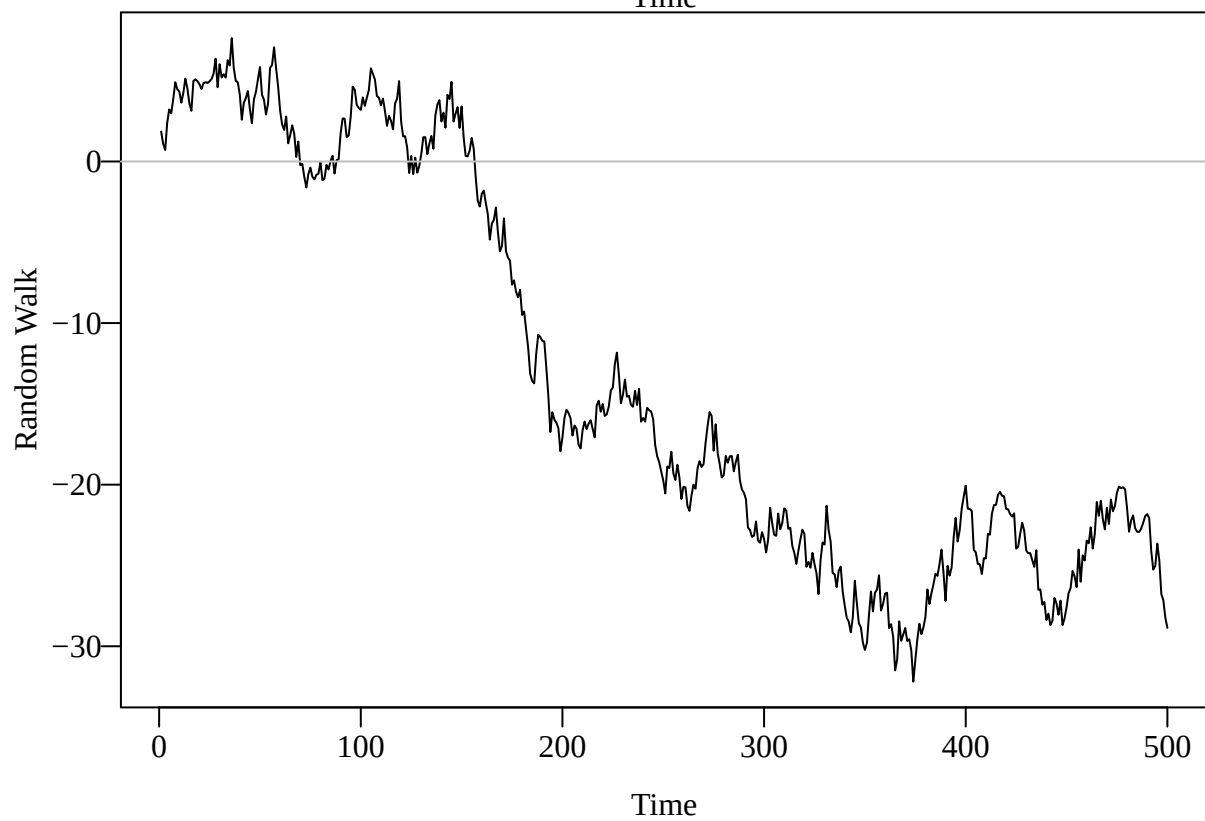
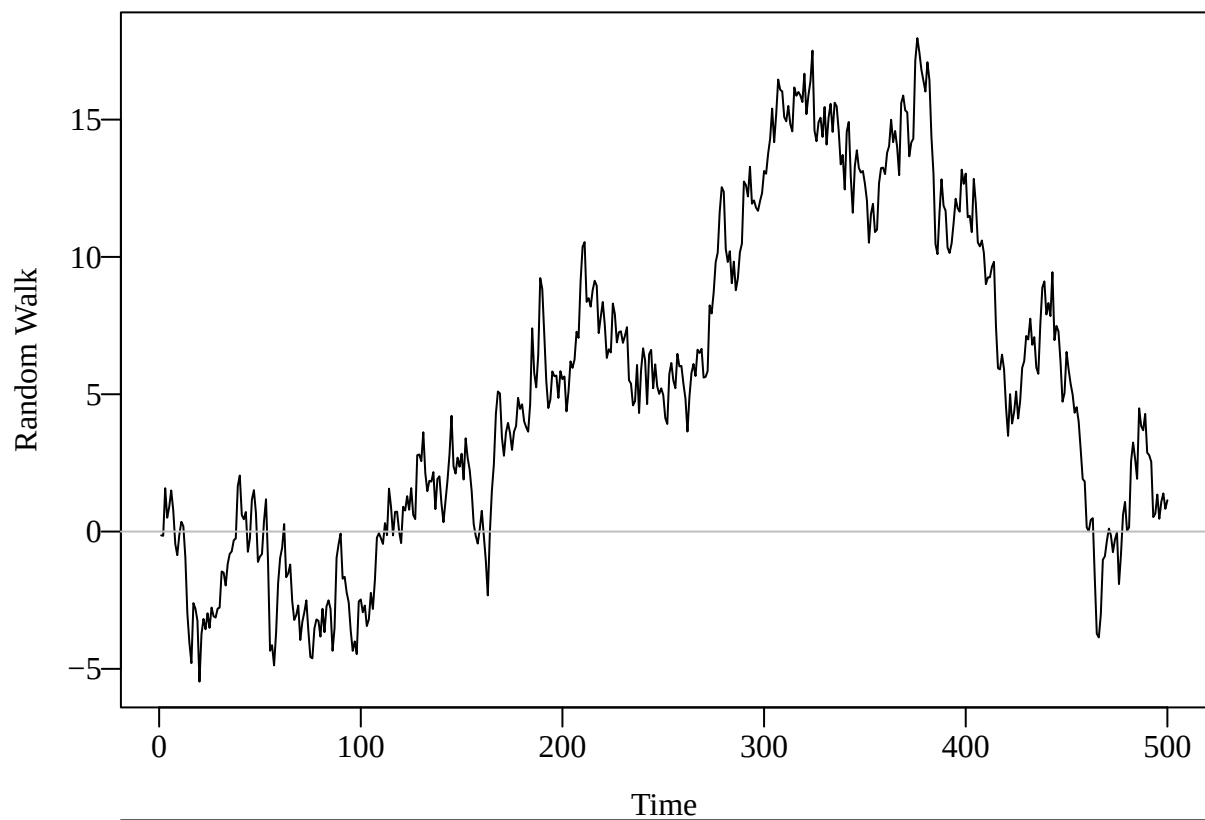
  # Generate 500 i.i.d. N(0, 1) innovations
  z <- rnorm(500)

  # Construct and plot the random walk as the cumulative sum of innovations
  plot(1:500, cumsum(z), type = "l", xlab = "Time",
      ylab = "Random Walk")

  # Add a horizontal reference line at zero
  abline(h = 0, col = "gray")
}
```







Gaussian process

Different covaraince functions (kernels)

```
library(fields)

# Commonly used covariance functions

# Define the exponential covariance function
cov.exp <- function(h, pars) pars[1] * exp(-h / pars[2])

# Define the Gaussian (squared exponential) covariance function
cov.doubleExp <- function(h, pars) pars[1] * exp(-(h / pars[2])^2)

# Define the Matérn covariance function
cov.Matern <- function(h, pars) Matern(h, phi = pars[1], range = pars[2],
                                         smoothness = pars[3])

# Create a sequence of temporal distances
xg <- seq(0, 5, 0.01)

# Evaluate the exponential covariance function
c_exp <- cov.exp(xg, c(1, 0.75))

# Evaluate the Gaussian covariance function
c_doubleExp <- cov.doubleExp(xg, c(1, 1))

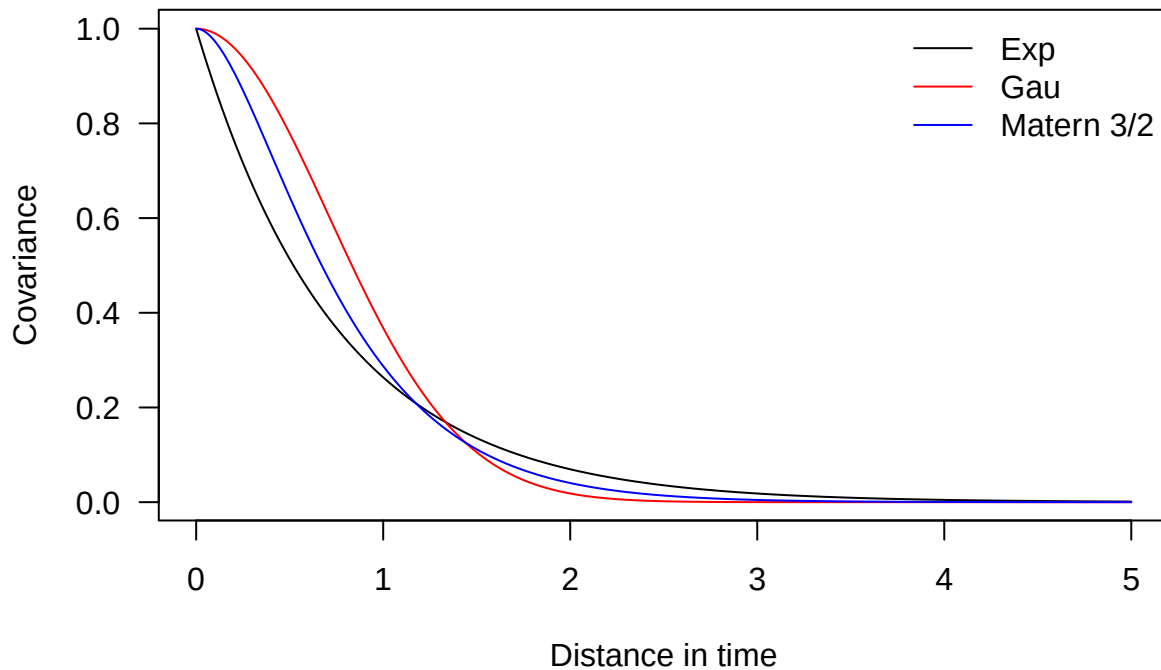
# Evaluate the Matérn covariance function with smoothness 3/2
c_Matern <- cov.Matern(xg, c(1, 0.4, 1.5))

# Plot the exponential covariance function
plot(xg, c_exp, type = "l", ylab = "Covariance", xlab = "Distance in time",
      las = 1)

# Add the Gaussian covariance function
lines(xg, c_doubleExp, col = "red")

# Add the Matérn covariance function
lines(xg, c_Matern, col = "blue")

# Add a legend identifying the three covariance functions
legend("topright", legend = c("Exp", "Gau", "Matern 3/2"),
      col = c("black", "red", "blue"), lty = 1, bty = "n")
```



Generate one sample from each Gaussian Process with different kernels

```
# Construct covariance matrices using the three covariance functions
Sigma_exp <- cov.exp(rdist(xg), c(1, 0.75))
Sigma_doubleExp <- cov.doubleExp(rdist(xg), c(1, 1))
Sigma_Matern <- cov.Matern(rdist(xg), c(1, 0.4, 1.5))

# Load the MASS package for multivariate normal simulation
library(MASS)

# Set the random seed for reproducibility
set.seed(1234)

# Simulate one Gaussian process realization with exponential covariance
sim_exp_1d <- mvrnorm(n = 1, rep(0, 501), Sigma_exp)

# Simulate one Gaussian process realization with Gaussian covariance
sim_doubleExp_1d <- mvrnorm(n = 1, rep(0, 501), Sigma_doubleExp)

# Simulate one Gaussian process realization with Matérn 3/2 covariance
sim_Matern_1d <- mvrnorm(n = 1, rep(0, 501), Sigma_Matern)

# Plot the realization with exponential covariance
plot(xg, sim_exp_1d, type = "l", ylim = range(sim_exp_1d, sim_doubleExp_1d,
                                              sim_Matern_1d), ylab = "y", las = 1)

# Add the realization with Gaussian covariance
lines(xg, sim_doubleExp_1d, col = "red")

# Add the realization with Matérn 3/2 covariance
```

```
lines(xg, sim_Matern_1d, col = "blue")

# Add a legend identifying the three covariance models
legend("topleft", legend = c("Exp", "Gau", "Matern 3/2"),
      col = c("black", "red", "blue"), lty = 1, bty = "n")
```

